



1. Anatomía de un TAD

Un TAD (Tipo Abstracto de Datos), define un conjunto de valores y las operaciones que se pueden realizar sobre ellos. Es abstracto ya que se enfoca en las operaciones (en cómo interactuar con los datos) y no necesita conocer los detalles de la representación interna o bien el cómo están implementadas sus operaciones. La especificación de un TAD indica qué hacen las operaciones y no cómo lo hacen.

Empecemos con un ejemplo sencillo:

```
TAD Punto {  
  obs x :  $\mathbb{R}$   
  obs y :  $\mathbb{R}$   
  
  proc igualdad(in  $P_1 : Punto$ , in  $P_2 : Punto$ ) : bool {  
    asegura {  $res = true \leftrightarrow \dots$  }  
  }  
  
  proc crearPunto(in  $x : \mathbb{R}$ , in  $y : \mathbb{R}$ ) :  $Punto$  {  
    asegura {  $\dots$  }  
  }  
  
  proc mover(inout  $p : Punto$ , in  $dx : \mathbb{R}$ , in  $dy : \mathbb{R}$ ) {  
    requiere {  $\dots$  }  
    asegura {  $\dots$  }  
  }  
  
  aux theta( $p : Punto$ ) :  $\mathbb{R} = \dots$   
  pred estaEnElOrigen( $p : Punto$ ) {  
     $\dots$   
  }  
}
```

Analicemos cada parte:

1.1. Nombre

La primera línea contiene la palabra TAD seguida del nombre del TAD, que debería sea declarativo, es decir, describir el problema a resolver. Luego del nombre tenemos la definición del TAD entre llaves:

```
TAD Punto {  
   $\dots$   
}
```

1.2. Observadores

```
obs x :  $\mathbb{R}$   
obs y :  $\mathbb{R}$ 
```

Los observadores permiten describir una instancia del TAD en un determinado momento en términos de tipos más conocidos. La especificación de las operaciones del TAD se escribe en función de los observadores antes y después de su ejecución. Es decir *asegura* deben especificar el valor de todos los observadores. Los tipos de datos que se pueden usar son los de especificación descriptos en el apunte del tema, más dos nuevos tipos complejos $\text{conj}\langle T \rangle$ y $\text{dicc}\langle K, V \rangle$. Todos estos tipos y sus operaciones están descriptos en el Anexo de este apunte.

1.3. Operaciones

Las operaciones de un TAD se especifican mediante nuestro lenguaje de especificación. Se debe indicar el nombre del procedimiento, la lista de parámetros con su nombre, tipo e indicar si son *in* o *inout*. Opcionalmente las operaciones pueden devolver un valor. Los parámetros pueden ser de cualquier tipo de nuestro lenguaje de especificación.

La lista de operaciones que indiquemos en el TAD representan el *contrato* o *interfaz* del TAD, y será lo que luego implementemos y lo que usen los *clientes* del TAD. Por convención, salvo las funciones que crean nuevas instancias del TAD, vamos a usar como primer parámetro una variable de tipo del TAD.

```

proc crearPunto(in x : ℝ, in y : ℝ) : Punto {
  asegura { ... }
}

proc mover(inout p : Punto, in dx : ℝ, in dy : ℝ) {
  requiere { ... }
  asegura { ... }
}

```

Cada función debe indicar la precondition y la postcondition (**requiere** y **asegura**) Como ya indicamos, los **requiere** y **asegura** van a hacer referencia a los observadores del TAD. Para referirnos al observador x de la instancia t usamos la notación $t.x$. Para hablar del valor inicial de un parámetro de tipo *inout* usamos metavariabes, refiriendo al tipo que estamos definiendo, no al observador, es decir, $T_0.x$, no $t.X_0$. Las metavariabes deben declararse en el **requiere**.

```

proc crearPunto(in x : ℝ, in y : ℝ) : Punto {
  asegura { res.x = x ∧ res.y = y }
}

proc mover(inout p : Punto, in dx : ℝ, in dy : ℝ) {
  requiere { p = P0 }
  asegura { p.x = P0.x + dx ∧ p.y = P0.y + dy }
}

```

Importante: Para que la especificación sea completa, tenemos que describir el valor de todos los observadores al salir de la operación. Recordemos que la implementación final del TAD sólo debe respetar la especificación y puede modificar cualquier cosa que no se defina. Por ejemplo, si tenemos un *proc* para mover sólo en el eje horizontal x , tenemos que decir que el observador y queda igual:

```

proc moverHoriz(inout p : Punto, in dx : ℝ) {
  requiere { p = P0 }
  asegura { p.x = P0.x + dx ∧ p.y = P0.y }
}

```

1.4. Igualdad

Como parte de la especificación de un TAD, es necesario definir una operación de *igualdad* para definir cuándo dos instancias de un TAD son iguales. Puede alcanzar con que todos los observadores de cada instancia sean iguales a los obs. correspondientes en la otra instancia. Es el caso de nuestro ejemplo: si dos puntos tienen las mismas coordenadas x y y , entonces son el mismo punto. La definición sería

```

proc igualdad(in  $P_1 : Punto$ , in  $P_2 : Punto$ ) : bool {
  asegura {  $res = true \leftrightarrow P_1.x = P_2.x \wedge P_1.y = P_2.y$  }
}

```

Otras veces la igualdad no es tan directa. Por ejemplo si queremos describir un número racional a partir de dos números enteros (numerador y denominador) tendremos muchas instancias que en realidad representan el mismo número (numerador 1 y denominador 2, num. 4 y den. 8, etc.). En este caso, la *igualdad* debería ser:

```

proc igualdad(in  $R_1 : Racional$ , in  $R_2 : Racional$ ) : bool {
  asegura {  $res = true \leftrightarrow R_1.numerador/R_1.denominador = R_2.numerador/R_2.denominador$  }
}

```

Para dos instancias t_1 y t_2 de un TAD, si $t_1 = t_2$, entonces toda operación aplicada a t_1 debe dar el mismo resultado que si la aplicamos a t_2 , es decir, ambas deben tener el mismo valor de salida y, si aparecen como inout, se debe mantener que $t_1 = t_2$

1.5. Funciones y predicados auxiliares

Para facilitar la especificación, es posible definir predicados y funciones auxiliares, usando nuestro lenguaje de especificación. Estos pueden usarse en los requiere y asegura de las operaciones. Por ejemplo:

```

aux theta( $p : Punto$ ) :  $\mathbb{R} = \arctan(p.y/p.x)$ 
aux rho( $p : Punto$ ) :  $\mathbb{R} = \sqrt{p.x^2 + p.y^2}$ 
proc rotar(inout  $p : Punto$ , in  $a : \mathbb{R}$ ) {
  requiere {  $p = P_0$  }
  asegura {
     $p.x = rho(P_0) * \cos(theta(P_0) + a) \wedge$ 
     $p.y = rho(P_0) * \sin(theta(P_0) + a)$ 
  }
}

```

2. TADs paramétricos

Muchas veces vamos a querer especificar un TAD que tome algún tipo genérico, es decir, indistinto para la especificación, como parámetro. Por ejemplo, podemos definir un conjunto que, como sabemos, va a contener elementos todos de un mismo tipo, y el comportamiento que queremos especificar va a ser el mismo cualquiera sea éste. Definiremos entonces un TAD $\text{Conjunto}\langle T \rangle$, donde T representa el tipo de los elementos. Luego, al utilizar el TAD reemplazaremos el tipo T por el tipo concreto que queramos ($\text{Conjunto}\langle \mathbb{Z} \rangle$, $\text{Conjunto}\langle Punto \rangle$, etc.). A modo de ejemplo, vemos aquí parte de una posible especificación del TAD $\text{Cola}\langle T \rangle$.

```

TAD Cola $\langle T \rangle$  {
  obs elems : seq $\langle T \rangle$ 
  ...
  proc encolar(inout  $c : \text{Cola}\langle T \rangle$ , in  $e : T$ ) {
    requiere {  $c = C_0$  }
    asegura {  $c.elems = C_0.elems ++ \langle e \rangle$  }
  }
  ...
}

```

3. Correctitud, modelado y legibilidad

La especificación de problemas es una herramienta para que quede claro qué debe hacer un programa o sistema. Es decir que una especificación debe estar pensada para que quien la lea sepa qué debe implementar a la hora de programar el problema. Con los TADs podemos especificar problemas y sistemas arbitrariamente complejos. Decimos que un TAD es un *modelo* de un problema, es decir, una representación abstracta del comportamiento que debe tener y la información que debe guardar un sistema que lo resuelva.

Un TAD “bien hecho” debería ser

- **Correcto:** Es decir, especificar el problema que queremos resolver, cubriendo todos los casos y las restricciones necesarias.
- **Legible:** Es decir, quien lee el TAD debería poder entender qué problema se está resolviendo sin dificultad.
- Un buen **Modelo** del problema: Es decir, representar la información sobre la que estamos especificando fielmente, sin introducir información extra

3.1. Declaratividad

Para que un TAD sea legible, los nombres que elegimos al definirlo deben ser declarativos, es decir, describir para qué se usa cada observador, procedimiento, predicado, etcétera. Es importante no sólo elegir bien los nombres de los TADs y de los observadores sino también usar *renombres de tipos*.

Por ejemplo, si queremos representar alumnos de una escuela por su número de documento o algún otro identificador numérico podemos escribir.

```
Alumne ES  $\mathbb{Z}$ 
```

y luego usarlos como cualquier tipo

```
proc calificarAlumne(inout e : Escuela, in a : Alumne, in nota :  $\mathbb{Z}$ ) {  
  ...  
}
```

Cuando necesitamos representar información que tiene varios valores asociados, es más declarativo usar **structs** con nombres en lugar de **tuplas**. Por ejemplo, si queremos representar el alumne con su DNI y nota promedio es más claro

```
Alumne ES struct  $\langle DNI : \mathbb{Z}, promedio : \mathbb{R} \rangle$   
  
proc promedio(in a : Alumne) :  $\mathbb{R}$  {  
  asegura { res = a.promedio }  
}
```

en lugar de

```
Alumne ES tupla  $\langle \mathbb{Z}, \mathbb{R} \rangle$   
  
proc promedio(in a : Alumne) :  $\mathbb{R}$  {  
  asegura { res = a1 }  
}
```

3.2. Modelado

Los observadores y operaciones de un TAD definen un *modelo* de un problema. Sólo con leerlos deberíamos poder tener una idea de qué es lo importante del problema, qué información está relacionada con cuál y cómo manipularla. Es importante tener esto en cuenta en el momento de la elección de los observadores y de los parámetros de los procedimientos.

Por ejemplo, si queremos representar los alumnos de una escuela, consideremos estos tres observadores alternativos que podríamos elegir:

- `obs alumnos : conj⟨Alumne⟩`
- `obs alumnos : seq⟨Alumne⟩`
- `obs alumnos : dicc⟨Alumne, Promedio⟩`

En el primer caso estamos modelando el alumnado como un grupo de alumnos sin que nos importe más que su DNI (o lo que sea que usamos para identificarlos). En el segundo caso, en cambio, estamos usando una estructura que define un orden¹ entre los alumnos. Finalmente en el tercer caso estamos asociando una nota promedio a cada alumno, con lo que estamos dando a entender que esa nota es importante dentro de nuestro sistema.

En principio ninguno de estos observadores es incorrecto: todos representan a los alumnos de la escuela. Sin embargo, son modelos diferentes, que pueden servir para representar mejor (o peor) diferentes problemas que trabajen con esa información.

Asimismo, consideremos estas opciones de `procs`:

- `proc abrirEscuela() : Escuela`
- `proc abrirEscuela(in alumnos : seq⟨Alumne⟩) : Escuela`
- `proc abrirEscuela(in alumnos : conj⟨Alumne⟩) : Escuela`

Las tres son formas válidas de crear el TAD, pero corresponden a modelos distintos. En el primer caso la escuela se “inicializa” sin alumnos (es razonable esperar un `proc inscribirAlumne(inout e : Escuela, in a : Alumne)`), en el segundo tenemos a todos los alumnos en algún orden, y en el último simplemente a todos los alumnos.

¹Además de permitir elementos repetidos, aunque eso se puede resolver en la especificación